

一.环境验证与基础验证

1.1 实验平台

操作系统：openKylin 2.0 SP2

内核版本：6.6.110-g89f7b81a4b04

目标架构：linux-riscv64

C 库版本：glibc 2.41

CPU 型号：QEMU RISC-V Virt(通用 rv64)

CPU 核心数：4

内存容量：3.8 GiB

磁盘可用空间：20G 总 / 12G 可用

1.2 安装 pixi

1.2.1 安装命令

```
curl -fsSL https://pixi.sh/install.sh | bash
```

```
source ~/.bashrc
```

```
pixi --version
```

1.2.2 安装结果

```
root@openkylin:~/pixi-riscv-demo# curl -fsSL https://pixi.sh/install.sh | bash
source ~/.bashrc
pixi --version
This script will automatically download and install Pixi (latest) for you.
Getting it from this url: https://github.com/prefix-dev/pixi/releases/latest/download/pixi-riscv64gc-unknown-linux-gnu.tar.gz
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0  0 --:--:--  0:00:01 --:--:--    0
  0     0    0     0    0     0      0  0 --:--:--  0:00:01 --:--:--    0
 3 32.1M    3 1229k    0     0 72354      0  0:07:46  0:00:17  0:07:29 53292^C
pixi 0.74.0
```

已经安装 pixi 0.74.0

1.3 项目初始化与配置

1.3.1 初始化项目

```
mkdir pixi-riscv-demo && cd pixi-riscv-demo
```

pixi init

1.3.2 pixi.toml 配置内容

channels = ["https://conda.openkylin.top/", "conda-forge"]platforms :

["linux-riscv64"], 指定目标平台

[system-requirements]: libc = "2.41", 声明系统 C 库版本。

1.4 Lockfile 生成验证

1.4.1 执行命令

pixi install

1.4.2 执行结果

```
root@openkylin:~/pixi-riscv-demo# pixi install
WARN Encountered 1 warning while parsing the manifest:
  ⚠ the `[system-requirements]` table is deprecated in favor of virtual packages on `platforms`
    [root/pixi-riscv-demo/pixi.toml:6:1]
5 |
6 | ┌─┐ [system-requirements]
7 | │ └─┐ libc = "2.41"
8 | │   └─┐ declare these on the `platforms` entries instead
  └─┘
help: e.g. platforms = [{ platform = "linux-64", cuda = "12" }]
✓ The default environment has been installed.
```

1.4.3 Lockfile 信息

文件名: pixi.lock

文件大小: 15 KB

锁定包数量: 91

平台标识: linux-riscv64-glibc-2-41

conda 通道可为 riscv64 提供基础工具链 (python、gcc、binutils、glibc 等共 91 个包), lockfile 正常生成, 依赖解析无冲突

1.5 对比试验: 安装 numpy

为验证 pixi 在 RISC-V 平台的包管理能力, 设计三组对照实验。

1.5.1 实验①：系统 pip 安装（失败）

命令：pip3 install numpy

输出：

```
root@openkylin:~/pixi-riscv-demo# pip3 install numpy
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have python3-full installed.

    If you wish to install a non-Debian packaged Python application,
    it may be easiest to use pipx install xyz, which will manage a
    virtual environment for you. Make sure you have pipx installed.

    See /usr/share/doc/python3.12/README.venv for more information.

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the risk of breaking your Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
```

失败原因：openKylin 遵循 PEP 668 规范，系统 Python 被标记为 externally-managed，禁止 pip 直接写入系统 site-packages。

1.5.2 实验②：pixi conda 通道安装（失败）

命令：time pixi add numpy

输出：

```
root@openkylin:~/pixi-riscv-demo# time pixi add numpy
WARN Encountered 1 warning while parsing the manifest:
  the '[system-requirements]' table is deprecated in favor of virtual packages on 'platforms'
  [root/pixi-riscv-demo/pixi.toml:6:1]
5  ┌
6  │ └─ [system-requirements]
7  │ └─ libc = "2.41"
8  │ └─ declare these on the 'platforms' entries instead
   └─ help: e.g. platforms = [{ platform = "linux-64", cuda = "12" }]

Error:   × failed to solve requirements of environment 'default' for platform 'linux-riscv64-glibc-2-41'
  └─ × failed to solve the environment
     └─ Cannot solve the request because of: No candidates were found for numpy *.

real    0m7.179s
user    0m7.245s
sys     0m1.520s
```

失败原因：conda-forge 及 openKylin conda 镜像均未提供 linux-riscv64 架构的 numpy 预编译包。pixi 在依赖解析阶段多线程遍历通道索引（user 时间略大于 real，即解析跑满 CPU）后确认无

候选，约 7 s 报错退出，未进入下载/编译环节。

1.5.3 实验③：pixi PyPI 源码编译（成功）

命令：pixi add gcc

time pixi add --pypi numpy

输出：

```
root@openkylin:~/pixi-riscv-demo# cd /root/pixi-riscv-demo
pixi add gcc 2>&1 | tail -5
time pixi add --pypi numpy 2>&1
8
┌
└ help: e.g. platforms = [{ platform = "linux-64", cuda = "12" }]
✓Added gcc >=14.3.0,<14.4
WARN Encountered 1 warning while parsing the manifest:
the '[system-requirements]' table is deprecated in favor of virtual packages on 'platforms'
┌
└ ┌ /root/pixi-riscv-demo/pixi.toml:6:1
  │
  │ [system-requirements]
  │ └─ libc = "2.41"
  │
  └ declare these on the 'platforms' entries instead
┌
└ help: e.g. platforms = [{ platform = "linux-64", cuda = "12" }]

-- updating pypi packages in 'default'
-- updating pypi packages in 'default'
-- updating pypi packages in 'default'
-- Preparing to update pypi packages in 'default'
└ in 'default'
┌
└ ✓Added numpy >=2.5.1, <3
┌
└ updating pypi packages in 'default'
┌
└ updating pypi packages

Added these as pypi-dependencies.
real    163m48.039s
user    499m35.508s
sys     27m29.740s
```

成功原因：pixi 检测到 conda 通道无候选包后，自动回退至 PyPI 通道，拉取 numpy 源码（sdist），使用环境内 gcc 14.3 + meson 构建系统完成本地编译。user/real $\approx$ 3.05 表明编译过程使用约 3 个 CPU 核心并行。

1.5.4 对比图

#	方式	命令	结果	耗时	说明
①	系统 pip	<code>pip3 install numpy</code>	✗ 拒绝	—	PEP 668 保护机制
②	pixi conda	<code>pixi add numpy</code>	✗ 无候选	7.2 s	conda 无 riscv64 包
③	pixi PyPI	<code>pixi add --pypi numpy</code>	✓ 成功	163 m 48 s	源码编译，约 3 核并行

1.6 运行验证

命令：pixi run python -c "

```

import numpy as np

print('numpy version:', np.__version__)

a = np.random.rand(3, 3)

print('随机矩阵:\n', a)

print('特征值:', np.linalg.eigvals(a))

print('✅ numpy 在 riscv64 上工作正常')

"

```

输出:

```

root@openkylin:~/pixi-riscv-demo# pixi run python -c "
import numpy as np
print('numpy version:', np.__version__)
a = np.random.rand(3, 3)
print('随机矩阵:\n', a)
print('特征值:', np.linalg.eigvals(a))
print('✅ numpy 在 riscv64 上工作正常')
"
WARN Encountered 1 warning while parsing the manifest:
⚠️ the `[system-requirements]` table is deprecated in favor of virtual packages on `platforms`
   [root/pixi-riscv-demo/pixi.toml:6:1]
5 |
6 | ┌─┐ [system-requirements]
7 | │ └─┐ libc = "2.41"
8 | │   └─┐ declare these on the `platforms` entries instead
   └─┬─┘
     help: e.g. platforms = [{ platform = "linux-64", cuda = "12" }]

numpy version: 2.5.1
随机矩阵:
[[0.92578631 0.09594235 0.89596347]
 [0.15098512 0.91831833 0.81349852]
 [0.99625065 0.8242309 0.72510239]]
特征值: [ 2.14461078+0.j  0.80045936+0.j -0.37586312+0.j]
✅ numpy 在 riscv64 上工作正常

```

✅ numpy 在 riscv64 上工作正常

结论:

numpy 2.5.1 在 openKylin RISC-V (linux-riscv64, glibc 2.41) 上可正常完成矩阵运算与线性代数计算, BLAS/LAPACK 后端工作正确。

二.

二. 基于 whisper.cpp 的 RISC-V 语音识别实验

2.1 实验目的

在 x86_64 主机上通过 QEMU 用户态模拟构建 RISC-V 64 位运行环境，验证 RISC-V 生态下 AI 推理工具链的可用性。交叉编译开源语音识别引擎 whisper.cpp，使其生成 RISC-V 64 位可执行文件，并在模拟环境中完成中英文语音识别全流程。验证 whisper.cpp 的 RISC-V Vector Extension（RVV）向量化加速路径在模拟环境下的功能正确性。采集推理耗时数据，分析编码（encode）与解码（decode）阶段的性能瓶颈，为后续真实硬件部署提供参考。

2.2 实验环境

2.2.1 硬件与操作系统

项目	配置
主机架构	宿主机 x86_64 / 目标（QEMU 模拟） openKylin riscv64
操作系统	openKylin（基于 Linux）
CPU 核心数	4（实验使用 4 线程）
内存	≥ 8 GB

2.2.2 软件工具链

工具	版本 / 说明
pixi	跨平台包管理器，用于管理交叉编译工具链
CMake	构建系统，配置交叉编译目标
RISC-V GCC 交叉编译器	riscv64-unknown-linux-gnu-gcc，由 pixi 环境提供
QEMU	qemu-riscv64，用户态（user-mode）RISC-V 64 位模拟器
whisper.cpp	开源 C/C++ 语音识别库（基于 OpenAI Whisper 模型）
模型文件	ggml-tiny.bin（Whisper tiny，77.11 MB，FP16 量化，ftype = 1）

2.3 交叉编译流程

2.3.1 工具链准备

使用 pixi 创建 RISC-V 交叉编译环境，安装 riscv64-unknown-linux-gnu-gcc 及相关工具链：

```
cd ~/pixi-riscv-demo/whisper.cpp
```

```
pixi shell # 激活 pixi 环境，加载 RISC-V 交叉编译器
```

2.3.2 CMake 交叉编译配置

通过 CMake 工具链文件指定目标架构为 RISC-V 64 位：

```
cmake -B build \
```

-DCMAKE_TOOLCHAIN_FILE=cmake/riscv64-toolchain.cmake \

-DCMAKE_BUILD_TYPE=Release \

-DWHISPER_RVV=ON

2.3.3 编译与产物验证

cmake --build build -j\$(nproc)

编译完成后，使用 file 命令验证产物架构：

file build/bin/whisper-cli

输出：ELF 64-bit LSB executable, UCB RISC-V, ...

确认生成的 whisper-cli 为 RISC-V 64 位 ELF 可执行文件，而非 x86_64。

2.4 QEMU 用户态模拟环境配置

2.4.1 运行方式

采用 QEMU 用户态模拟（user-mode emulation），无需启动完整 RISC-V 操作系统，直接在 x86_64 主机上运行 RISC-V ELF 二进制：

export LD_LIBRARY_PATH=\$PWD/build/bin:\$LD_LIBRARY_PATH

qemu-riscv64 ./build/bin/whisper-cli [参数]

2.4.2 动态库路径

whisper-cli 依赖同目录下的 libwhisper.so，

需设置：

export LD_LIBRARY_PATH=\$PWD/build/bin:\$LD_LIBRARY_PATH

2.5 英文语音识别实验（基线）

2.5.1 实验输入

项目	值
音频文件	<code>samples/jfk.wav</code>
内容	JFK 肯尼迪总统就职演讲片段
采样率	16 kHz
声道	单声道 (mono)
时长	11.0 秒 (176,000 samples)
语言参数	<code>-l en</code> (英语)

2.5.2 执行命令

```
./build/bin/whisper-cli -m models/ggml-tiny.bin -f samples/jfk.wav -l en  
-t 4
```

参数说明:

参数	含义
<code>-m</code> <code>models/ggml-tiny.bin</code>	指定 Whisper tiny 模型

-f samples/jfk.wav	输入音频文件
-l en	指定识别语言为英语
-t 4	使用 4 个 CPU 线程

2.5.3 识别结果

[00:00:00.000 --> 00:00:07.940] And so my fellow Americans ask not
what your country can do for you

[00:00:07.940 --> 00:00:10.760] ask what you can do for your
country.

```
[00:00:00.000 --> 00:00:07.940] And so my fellow Americans ask not what your country can do for you
[00:00:07.940 --> 00:00:10.760] ask what you can do for your country.
```

2.5.4 性能数据

阶段	耗时	占比	说明
load time	3,135.67 ms	0.9%	模型加载（77.11 MB）
mel time	1,185.17 ms	0.3%	音频 → 80 维梅尔频谱
encode time	271,562.69 ms	75.3%	音频编码器（4 层 Transformer）
sample	3,002.34 ms	0.8%	138 次采样，21.76 ms/次

time			
batchd	80,556.44 ms	22.3%	136 次解码批处理，592.33 ms/次
time			
decode	0.00 ms	—	（含在 batchd 中）
time			
total time	360,733.69 ms（ $\approx$ 6.0 min）	100%	端到端总耗时

实时率（RTF）= $360.7\text{ s} \div 11.0\text{ s} \approx 32.8\times$

2.6 中文语音识别实验

2.6.1 实验输入

项目	值
音频文件	/root/chinese.wav
来源	ModelScope 语音识别示例音频
采样率	16 kHz
声道	单声道

时长	13.1 秒 (208,832 samples)
----	---------------------------

语言参数	-l zh (中文)
------	------------

2.6.2 执行命令

```
./build/bin/whisper-cli -m models/ggml-tiny.bin -f /root/chinese.wav -l zh -t 4
```

2.6.3 识别结果

[00:00:00.000 --> 00:00:02.340] 正是因為存在絕對證明

[00:00:02.340 --> 00:00:04.660] 所以我們接受現實的相對證

[00:00:04.660 --> 00:00:07.500] 但是不要因為現實的相對證明

[00:00:07.500 --> 00:00:10.060] 我們就認為這個世界沒有證明

[00:00:10.060 --> 00:00:13.100] 因為如果當你認為這個世界沒有證明

```
[00:00:00.000 --> 00:00:02.340] 正是因為存在絕對證明
[00:00:02.340 --> 00:00:04.660] 所以我們接受現實的相對證
[00:00:04.660 --> 00:00:07.500] 但是不要因為現實的相對證明
[00:00:07.500 --> 00:00:10.060] 我們就認為這個世界沒有證明
[00:00:10.060 --> 00:00:13.100] 因為如果當你認為這個世界沒有證明
```

注：输出为繁体中文，这是 Whisper tiny 模型的已知特性（tiny 模型的中文 tokenizer 倾向输出繁体），并非识别错误。使用 base 或 small 以上模型可显著改善。

2.6.4 性能数据

阶段	耗时	占比	说明
load time	2,715.59 ms	0.6%	模型加载
mel time	1,303.04 ms	0.3%	梅尔频谱提取
encode time	260,898.17 ms	57.1%	音频编码器
sample time	6,816.91 ms	1.5%	316 次采样，21.57 ms/次
batchd time	183,684.89 ms	40.2%	313 次解码批处理，586.85 ms/次
decode time	586.10 ms	0.1%	解码后处理
total time	457,192.38 ms (≈ 7.6 min)	100%	端到端总耗时

实时率（RTF）= 457.2 s ÷ 13.1 s ≈ 34.9×

2.7 性能对比与分析

2.7.1 中英文对比总表

指标	英文 (jfk.wav)	中文 (chinese.wav)	差异分析
----	--------------	------------------	------

音频时长	11.0 s	13.1 s	中文音频略长
total time	360.7 s (6.0 min)	457.2 s (7.6 min)	中文慢 26.8%
encode time	271.6 s (75.3%)	260.9 s (57.1%)	基本持平 (与语言无关)
batchd time	80.6 s / 136 runs	183.7 s / 313 runs	中文是英文的 2.3 倍
sample time	3.0 s / 138 runs	6.8 s / 316 runs	中文 token 数多
解 码 token 数	138	316	中文 BPE 分词粒度更细
RTF	32.8×	34.9×	基本一致
RVV 加 速	☒ RISC_V = 1	☒ RISC_V = 1	均启用

2.7.2 瓶颈分析

(1) 编码阶段 (encode) 是绝对瓶颈

两次实验中, encode 耗时均超过 260 秒, 占总耗时的 57%–75%。

Whisper 的音频编码器包含 4 层 Transformer (n_audio_layer = 4), 需对 1500 帧梅尔频谱 (n_audio_ctx = 1500) 做全序列自注意力计算,

计算复杂度为 $O(n^2)$ ，是推理耗时的主要来源。

(2) 中文解码开销显著高于英文

中文 decode 阶段生成 316 个 token，是英文（138 个）的 2.3 倍。这是因为 Whisper 的 BPE（Byte Pair Encoding）分词器对中文的分词粒度更细——英文一个单词通常为 1–2 个 token，而中文一个汉字可能需要 2–3 个 token。token 数增加直接导致：

batchd 批处理次数增加（313 vs 136）

sample 采样次数增加（316 vs 138）

解码总耗时增加（183.7 s vs 80.6 s）

(3) 编码阶段与语言无关

encode 耗时在两次实验中基本持平（271.6 s vs 260.9 s），因为编码器仅处理固定维度的梅尔频谱（80 维 × 1500 帧），不涉及语言信息。语言差异仅影响解码器阶段。

2.7.3 模型参数摘要

运行时日志输出的模型结构参数：

参数	值	说明
n_vocab	51,865	词表大小（含 1,608 个额外 token）
n_audio_ctx	1,500	音频上下文窗口（帧数）
n_audio_state	384	编码器隐藏层维度

n_audio_head	6	编码器注意力头数
n_audio_layer	4	编码器 Transformer 层数
n_text_ctx	448	文本上下文窗口
n_text_state	384	解码器隐藏层维度
n_text_head	6	解码器注意力头数
n_text_layer	4	解码器 Transformer 层数
n_mels	80	梅尔频谱维度
n_langs	99	支持语言数
ftype	1	FP16 半精度
model size	77.11 MB	模型文件大小